ANALISIS KOMPARATIF PERFORMA DIRECT STATE UPDATE MVI DAN REDUCER-BASED MVI PADA APLIKASI ANDROID BERBASIS JETPACK COMPOSE

Disusun oleh:
Audy Fitrah Habibie
NIM: 235150400111018



PROGRAM STUDI SISTEM INFORMASI
DEPARTEMEN SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026

DAFTAR ISI

DAFTAR ISI ii
DAFTAR TABEL iv
DAFTAR GAMBAR v
DAFTAR LAMPIRAN vi
BAB 1 PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 3
1.3 Tujuan 3
1.4 Manfaat 3
1.5 Batasan Masalah 4
1.6 Sistematika Pembahasan 4
BAB 2 LANDASAN KEPUSTAKAAN 6
2.1 Tinjauan Pustaka 6
2.2 Dasar Teori 8
2.2.1 Jetpack Compose dan Declarative UI 8
2.2.2 Pengelolaan State pada Android 9
2.2.3 Arsitektur Model-View-Intent (MVI) 9
2.2.4 Direct State Update MVI 9
2.2.5 Reducer-Based MVI 10
2.2.6 Metrik Responsivitas 10
2.2.7 Metrik Penggunaan CPU 10
BAB 3 METODOLOGI 12
3.1 Tipe Penelitian 12
3.2 Jenis dan Rancangan Penelitian 12
3.3 Skenario Eksperimen 13
3.4 Metode dan Metrik Pengukuran 15
3.4.1 Pengukuran Responsivitas 15
3.5.2 Pengukuran Penggunaan CPU 16
3.5 Lingkungan dan Instrumen Pengujian 16

3.6 Teknik Analisis Data 16
DAFTAR REFERENSI 18

DAFTAR TABEL

Tabel 2.1 Penelitian Terdahulu 7
Tabel 3.1 Klasifikasi Tingkat State 13
Tabel 3.2 Skenario Eksperimen 14
Tabel 3.3 Hipotesis Penelitian 15

DAFTAR GAMBAR

Gambar 3.1 Tahapan Metode Penelitian 12
Gambar 3.2 Hasil Pilot Test 13
Gambar 3.3 Dua Tipe MVI 14

DAFTAR LAMPIRAN

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Sistem operasi Android terus mendominasi pasar perangkat seluler secara global dengan pangsa pasar yang konsisten berada pada kisaran 70 hingga 73 persen hingga tahun 2025 (CommandLinux, 2026). Skala adopsi yang masif tersebut mendorong transformasi signifikan dalam rekayasa antarmuka pengguna (user interface), salah satunya melalui transisi dari pendekatan imperatif berbasis tata letak XML menuju paradigma deklaratif modern menggunakan Jetpack Compose (Kwiatkowski and Smołka, 2025). Adopsi Jetpack Compose telah diterapkan secara luas pada berbagai aplikasi skala besar di Google Play Store guna membangun antarmuka yang adaptif dan terstruktur (Android Developers, 2025). Berbeda dengan paradigma imperatif yang memanipulasi hierarki tampilan secara langsung, Jetpack Compose mendeskripsikan antarmuka sebagai representasi langsung dari kondisi atau state aplikasi. Ketika state mengalami pembaruan, sistem secara otomatis mengeksekusi proses recomposition pada komponen antarmuka yang relevan guna merefleksikan perubahan data terkini (Android Developers, 2025). Mekanisme ini menjadikan efisiensi pengelolaan alur transisi state sebagai faktor arsitektural yang sangat fundamental dalam menentukan stabilitas dan responsivitas aplikasi secara menyeluruh.

Untuk mengelola aliran data pada sistem deklaratif tersebut, arsitektur *Model-View-Intent* (MVI) semakin banyak diadopsi oleh komunitas pengembang (Vijaywargi dan Boddapati, 2024). Keunggulan utama MVI terletak pada penerapan prinsip aliran data satu arah (Unidirectional Data Flow) dan prinsip sumber kebenaran tunggal (Single Source of Truth) (Chauhan et al., 2021). Pada pola ini, setiap interaksi pengguna direpresentasikan secara eksplisit sebagai *Intent*, yang kemudian diproses untuk menghasilkan representasi *UiState* baru yang bersifat *immutable* sebelum dikirimkan kembali ke antarmuka (Vijaywargi dan Boddapati, 2024).

Kendati sama-sama menerapkan prinsip *Unidirectional Data Flow* (UDF), implementasi mekanisme transisi dari *Intent* menuju *State* baru dalam arsitektur MVI dapat dilakukan melalui pendekatan yang berbeda, di antaranya *Direct State Update MVI* dan *Reducer-Based MVI* (Kostadinov, 2025). Pada pendekatan *Direct State Update MVI*, *Intent* yang diterima oleh *ViewModel* diproses oleh fungsi pengendali (*handler*) untuk menjalankan logika dan memperbarui *state* secara langsung (Kostadinov, 2025). Sementara itu, *Reducer-Based MVI* mengadopsi konsep *reducer* yang berakar dari paradigma pemrograman fungsional dan dipopulerkan melalui arsitektur pengelolaan *state* seperti Redux (Redux, 2021; Boduch dan Derks, 2023). Pada pendekatan ini, perubahan *state* dikelola melalui fungsi *reducer* yang terpusat untuk menghasilkan *state* baru berdasarkan kondisi *state* dan aksi yang diterima (Vijaywargi dan Boddapati, 2024). Mekanisme tersebut memberikan struktur transisi *state* yang lebih terorganisasi dan dapat mendukung keterlacakan perubahan kondisi serta *maintainability* kode, terutama

ketika kompleksitas logika aplikasi meningkat (Kostadinov, 2025; zen8labs, 2025). Pendekatan Reducer-Based MVI kerap dipilih dan direkomendasikan dalam literatur teknis maupun praktisi industri untuk modul fitur dengan kompleksitas logika bisnis tingkat menengah hingga tinggi, terutama ketika pengembang membutuhkan pemisahan yang tegas antara lapisan antarmuka dan logika bisnis demi mencapai predictability dan maintainability yang lebih baik (Kostadinov, 2025; zen8labs, 2025). Namun, penerapan mekanisme *reducer* juga menambahkan tahapan pemrosesan dalam siklus pembaruan *state* dibandingkan pendekatan pembaruan secara langsung.

Meskipun menawarkan struktur pengelolaan *state* yang lebih terpusat dan dapat mendukung kemudahan pemeliharaan kode, pendekatan *Reducer-Based MVI* memiliki tahapan pemrosesan yang lebih panjang dibandingkan *Direct State Update MVI*, karena setiap intent harus melewati fungsi reducer sebelum state baru dihasilkan. Penambahan tahapan pemrosesan ini menjadi hal yang perlu diperhatikan karena setiap proses tambahan dapat memberikan beban komputasi tambahan pada aplikasi. Beban komputasi tersebut akan memerlukan sumber daya pemrosesan CPU, sehingga peningkatan beban komputasi berpotensi meningkatkan penggunaan CPU selama aplikasi menjalankan proses tersebut. Fu et al. (2022) melalui eksperimen terkontrol pada perangkat Android menunjukkan bahwa CPU merupakan faktor penting yang memengaruhi responsivitas antarmuka. Peningkatan beban komputasi dapat meningkatkan waktu respons UI secara signifikan, sehingga perubahan pada mekanisme pemrosesan aplikasi perlu ditinjau tidak hanya dari struktur dan pemeliharaan kode, tetapi juga dari dampaknya terhadap responsivitas.

Permasalahan tersebut menjadi relevan ketika pengembang perlu membangun aplikasi yang memiliki struktur kode yang terorganisasi dan mudah dipelihara tanpa mengabaikan responsivitas aplikasi. Manfaat tersebut perlu dipertimbangkan bersamaan dengan kemungkinan adanya tambahan pekerjaan pemrosesan dibandingkan pembaruan *state* secara langsung. Dengan demikian, pemilihan pendekatan pengelolaan *state* tidak hanya berkaitan dengan bagaimana kode dapat disusun dan dipelihara, tetapi juga perlu mempertimbangkan implikasinya terhadap performa aplikasi. Penelitian ini dilakukan untuk memberikan bukti empiris mengenai perbandingan tersebut, sehingga pengembang dapat memperoleh gambaran mengenai karakteristik dari masing-masing pendekatan ketika menerapkan mekanisme pengelolaan *state* yang berbeda.

MVI sering digunakan pada aplikasi yang memiliki *state* kompleks, seperti *marketplace* dan media sosial, karena pengelolaan *state* yang terstruktur dapat membantu menjaga *maintainability*. Namun, masih terdapat kendala responsivitas pada aplikasi dengan karakteristik tersebut. Hal ini terlihat dari pengembang yang mempertanyakan rekomendasi arsitektur karena halaman kompleks pada aplikasinya masih mengalami masalah responsivitas (GitHub Community, 2026). Permasalahan serupa juga ditemukan pada fitur *shopping cart* pada aplikasi *marketplace*, yang dikeluhkan pengguna karena mengalami tampilan

tersendat (Reddit, 2024). Selain itu, kendala responsivitas juga masih dilaporkan pada aplikasi media sosial (Samsung Community, 2025). Kondisi tersebut menunjukkan bahwa peningkatan *maintainability* melalui pengelolaan *state* yang lebih terstruktur perlu dipertimbangkan bersama dengan potensi dampaknya terhadap performa, sehingga trade-off antara keduanya menjadi penting untuk dikaji pada mekanisme MVI yang berbeda.

Berdasarkan latar belakang tersebut, penelitian ini berfokus pada perbandingan responsivitas antara *Direct State Update MVI* dan *Reducer-Based MVI* pada aplikasi Android berbasis Jetpack Compose. Penelitian ini diharapkan dapat memberikan bukti empiris mengenai konsekuensi performa dari perbedaan mekanisme transisi *state* tersebut. Hasil penelitian tersebut dapat membantu memperjelas apakah penggunaan mekanisme *reducer* yang ditujukan untuk memberikan struktur pengelolaan *state* yang lebih terorganisasi dan mendukung *maintainability* juga memiliki karakteristik responsivitas yang berbeda dibandingkan pendekatan pembaruan *state* secara langsung. Dengan demikian, pengembang dapat mempertimbangkan aspek maintainability dan responsivitas secara bersamaan ketika memilih pendekatan pengelolaan *state* pada aplikasi Jetpack Compose.

1.2 Rumusan Masalah

1. Bagaimana perbandingan responsivitas antara Direct State Update MVI dan Reducer-Based MVI pada aplikasi Android berbasis Jetpack Compose?
2. Bagaimana perbandingan penggunaan CPU antara Direct State Update MVI dan Reducer-Based MVI pada aplikasi Android berbasis Jetpack Compose?

1.3 Tujuan

1. Mengukur dan membandingkan responsivitas antara Direct State Update MVI dan Reducer-Based MVI pada aplikasi Android berbasis Jetpack Compose.
2. Mengukur dan membandingkan penggunaan CPU antara Direct State Update MVI dan Reducer-Based MVI pada aplikasi Android berbasis Jetpack Compose.
3. Menentukan pendekatan pengelolaan state, antara Direct State Update MVI atau Reducer-Based MVI, yang memberikan karakteristik performa lebih baik berdasarkan hasil pengukuran responsivitas, dan penggunaan CPU.

1.4 Manfaat

Hasil penelitian ini diharapkan dapat memberikan landasan empiris dan panduan praktis yang jelas bagi pengembang perangkat lunak dalam menentukan strategi pengelolaan state yang paling optimal saat membangun fitur aplikasi interaktif berbasis Jetpack Compose.

Pemanfaatan hasil penelitian ini akan membantu pengembang merancang aplikasi Android yang mampu menyeimbangkan aspek kemudahan pemeliharaan kode dengan efisiensi performa komputasi, sehingga kelancaran tampilan visual dan responsivitas antarmuka aplikasi tetap terjaga bagi kenyamanan pengguna.

Hasil pengukuran empiris menggunakan instrumen AndroidX Macrobenchmark, yang meliputi data responsivitas dan penggunaan CPU, dapat menjadi referensi metodologis bagi penelitian selanjutnya yang mengkaji performa pola arsitektur pengelolaan state pada aplikasi Android berbasis Jetpack Compose.

1.5 Batasan Masalah

Penelitian ini hanya membandingkan dua pendekatan pengelolaan state pada pola arsitektur Model-View-Intent (MVI), yaitu Direct State Update MVI dan Reducer-Based MVI, dan tidak mencakup pola arsitektur pengelolaan state lain di luar kedua pendekatan tersebut.

Aspek performa yang diukur dan dibandingkan dibatasi pada responsivitas, dan penggunaan CPU. Aspek-aspek lain di luar kedua metrik tersebut, seperti kemudahan atau keringkasan penulisan kode (developer experience), maintainability atau kemudahan pemeliharaan kode, skalabilitas tim pengembang, maupun kualitas kode secara umum, tidak menjadi fokus dan tidak dibahas dalam penelitian ini.

Pengujian dilakukan pada modul transaksi keranjang belanja berbasis Jetpack Compose menggunakan instrumen AndroidX Macrobenchmark, sehingga hasil penelitian ini terbatas pada konteks skenario dan beban kerja yang diujikan

1.6 Sistematika Pembahasan

Skripsi ini disusun secara sistematis agar memudahkan pembaca memahami alur penelitian dari awal hingga akhir. Adapun sistematika pembahasan terdiri dari:

BAB I PENDAHULUAN

Bab ini menguraikan latar belakang penelitian mengenai perbedaan mekanisme pengelolaan state pada Direct State Update MVI dan Reducer-Based MVI dalam aplikasi Android berbasis Jetpack Compose, rumusan masalah, tujuan penelitian, manfaat penelitian, batasan masalah, serta sistematika pembahasan skripsi secara keseluruhan.

BAB II LANDASAN KEPUSTAKAAN

Bab ini membahas teori, konsep, dan penelitian terdahulu yang relevan dengan penelitian, meliputi Jetpack Compose dan Declarative UI, pengelolaan state pada Android, arsitektur Model-View-Intent (MVI), Direct State Update MVI, Reducer-Based MVI, responsivitas, serta penggunaan CPU. Bab ini juga membahas penelitian terdahulu yang menjadi dasar dan pembanding dalam penelitian.

BAB III METODOLOGI PENELITIAN

Bab ini menjelaskan jenis dan rancangan eksperimen, pemodelan state dan struktur data, skenario beban kerja, metode dan metrik pengukuran, lingkungan dan instrumen pengujian, prosedur pelaksanaan eksperimen, serta teknik analisis data yang digunakan untuk membandingkan responsivitas dan penggunaan CPU pada Direct State Update MVI dan Reducer-Based MVI.

BAB IV HASIL

Bab ini menyajikan hasil pengujian penelitian berupa data pengukuran responsivitas dan penggunaan CPU pada Direct State Update MVI dan Reducer-Based MVI.

BAB V PEMBAHASAN

Bab ini membahas dan menginterpretasikan hasil penelitian dengan membandingkan responsivitas dan penggunaan CPU dari kedua pendekatan MVI serta mengaitkannya dengan rumusan masalah, hipotesis, dan pustaka.

BAB VI PENUTUP

Bab ini merupakan bagian akhir dari laporan penelitian yang berisi kesimpulan dan saran. Kesimpulan merangkum jawaban atas rumusan masalah berdasarkan hasil dan pembahasan yang telah diuraikan pada bab sebelumnya. Sementara itu, saran memberikan masukan untuk penelitian selanjutnya yang berkaitan dengan evaluasi performa pengelolaan state pada aplikasi Android berbasis Jetpack Compose.

BAB 2 LANDASAN KEPUSTAKAAN

2.1 Tinjauan Pustaka

Beberapa penelitian terdahulu yang relevan dengan penelitian ini diringkas sebagai berikut. Vijaywargi dan Boddapati (2024) membandingkan pola arsitektur MVP, MVVM, dan MVI, lalu menemukan bahwa MVI dengan aliran data satu arah (*unidirectional data flow*) dan *immutable state* efektif menekan *bug* berbasis *state* hingga 50 persen, meskipun menambah kompleksitas kode. Paramadani et al. (2023) menganalisis dampak pola arsitektur MVC dan MVVM terhadap waktu *rendering* antarmuka (*UI rendering time*), lalu menemukan bahwa MVVM secara signifikan menghasilkan waktu *rendering* UI yang lebih cepat berkat pemisahan logika bisnis dari *layer* antarmuka serta dukungan fitur *data binding*. Chauhan et al. (2021) menganalisis performa pemrosesan *state* pada MVI berbasis Kotlin Coroutines dibandingkan RxJava, dan menemukan bahwa Coroutines memproses *state* jauh lebih cepat melalui *lock-free channels* serta jauh lebih hemat alokasi memori dibanding utas JVM tradisional. Kwiatkowski dan Smołka (2025) menguji performa komponen Jetpack Compose menggunakan AndroidX Macrobenchmark, dan menemukan bahwa komponen spesialisasi (LazyColumn) jauh lebih stabil pada dataset besar dibanding komponen umum (Column). Lin et al. (2024) menginvestigasi penyebab penurunan responsivitas Android dan membuktikan bahwa penurunan kinerja antarmuka dominan dipicu oleh cacat desain perangkat lunak (*software glitching*), bukan oleh penuaan perangkat keras. Sementara itu, Chanddru (2025) mengulas optimasi performa teknis aplikasi mobile dan mendokumentasikan bahwa keterlambatan respons di atas 100 ms mulai terdeteksi oleh pengguna.

Penelitian-penelitian tersebut memiliki persamaan dengan penelitian ini dalam hal fokus pengkajian, yaitu evaluasi performa aplikasi Android melalui metrik latensi responsivitas, dan efisiensi manajemen *state*. Perbedaannya, Vijaywargi dan Boddapati (2024) hanya mengulas arsitektur MVI secara konseptual makro tanpa eksperimen komparatif antar-varian internal; Chauhan et al. (2021) membandingkan *framework* konkurensi (Coroutines vs RxJava) pada MVI tanpa melibatkan kerangka kerja UI deklaratif; Kwiatkowski dan Smołka (2025) menguji komponen fisik antarmuka Compose, bukan struktur logika pembaruan *state* di belakangnya; serta Lin et al. (2024), Chanddru (2025) berfokus pada optimasi tingkat platform dan persepsi pengguna secara umum. Kaitannya dengan rumusan masalah: karena belum ada penelitian empiris yang secara spesifik membandingkan efisiensi *Direct State Update MVI* dan *Reducer-Based MVI* pada lingkungan Jetpack Compose dalam hal responsivitas dan penggunaan CPU, pertanyaan penelitian ini belum terjawab oleh penelitian terdahulu dan karenanya perlu dijawab melalui penelitian ini. Perbandingan penelitian terdahulu dengan penelitian ini dirangkum pada Tabel 2.1.

Tabel 2.1 Penelitian Terdahulu

Penelitian	**Fokus**	**Hasil**	**Kaitan dengan penelitian ini**
Vijaywargi & Boddapati (2024	Analisis komparatif konseptual pola arsitektur MVP, MVVM, dan MVI pada aplikasi Android.	MVI dengan *Unidirectional Data Flow* (UDF) menekan *bug state* hingga 50%, namun menambah *boilerplate*.	Landasan teori komponen MVI. Penelitian ini memperdalam pengujian pada dua variasi transisi *state* internal MVI (*Direct* vs *Reducer*).
Paramadani et al. (2023)	Analisis komparatif dampak pola arsitektur MVC dan MVVM terhadap *UI rendering time* pada aplikasi navigasi Android.	MVVM menghasilkan waktu *rendering UI* secara signifikan lebih cepat dibanding MVC berkat pemisahan logika bisnis dan fitur *data binding*.	Landasan empiris bahwa pemisahan logika memengaruhi *rendering time*. Penelitian ini menguji spesifik dua varian *state update* internal MVI pada Jetpack Compose.
Chauhan et al. (2021)	Analisis performa pemrosesan *state* MVI berbasis Kotlin Coroutines vs RxJava.	Coroutines memproses *state* jauh lebih cepat via *lock-free channels* dan sangat hemat memori (ratusan bita vs 1 MB).	Acuan pemrosesan *state* berbasis Coroutines. Penelitian ini memperluas pengujian hingga pengukuran responsivitas UI Compose.
Kwiatkowski & Smołka (2025)	Evaluasi performa komponen Jetpack Compose (LazyColumn vs Column) via Macrobenchmark.	LazyColumn stabil pada dataset besar, sedangkan Column mengalami lonjakan memori dan latensi ekstrim.	Acuan metodologi Macrobenchmark untuk mengukur dampak pembaruan *state* MVI pada antarmuka Compose.
Lin et al. (2024)	Investigasi pemicu responsivitas buruk Android (*Slow Rendering*,	Penurunan responsivitas dominan dipicu oleh *software glitching* dan *redundant UI*	Landasan ambang batas *Slow Rendering* (>16,67 ms) dan bukti bahwa pemborosan

	Frozen Frames, ANR, SNR).	*rendering*, bukan penuaan *hardware*.	sumber daya berakar dari desain *software*.
Chanddru (2025)	Tinjauan dampak optimasi performa aplikasi mobile terhadap kepuasan dan retensi pengguna.	Latensi >100 ms mulai terdeteksi pengguna; alokasi RAM >150 MB pada RAM 2 GB meningkatkan *crash rate* 32%.	Ambang batas toleransi latensi dan memori dari perspektif pengalaman pengguna sebagai dasar urgensi penelitian.
Fu et al. (2022)	Studi eksperimental pengaruh utilisasi sumber daya (CPU, Memori, I/O) terhadap responsivitas UI via *adb input tap*	Persaingan CPU (*CPU competition*) adalah penyebab utama *laggy-UI*; penggunaan memori tidak berdampak langsung kecuali mendekati 100%.	Melandasi pentingnya mengevaluasi responsivitas antarmuka akibat efisiensi beban kerja utas CPU pada varian MVI.
Penelitian Ini	Evaluasi komparatif *Direct State Update MVI* vs *Reducer-Based MVI* pada Jetpack Compose via Macrobenchmark di perangkat fisik Android.	*(Rencana Hasil)* Menentukan varian MVI paling efisien berdasarkan responsivitas dan penggunaan CPU	Mengisi *research gap* dengan membandingkan mekanisme transisi *state* internal MVI secara spesifik pada lingkungan Jetpack Compose.

2.2 Dasar Teori

2.2.1 Jetpack Compose dan Declarative UI

Jetpack Compose adalah toolkit modern yang direkomendasikan oleh Google untuk membangun antarmuka pengguna (user interface) Android secara native dengan paradigma deklaratif (Android Developers, 2025). Berbeda dengan pendekatan imperatif berbasis tata letak XML yang memanipulasi hierarki View secara langsung melalui pemanggilan method, pendekatan deklaratif mendeskripsikan tampilan antarmuka berdasarkan kondisi atau state saat ini (Kwiatkowski and Smołka, 2025).

Mekanisme inti dalam Jetpack Compose adalah proses recomposition, yaitu proses pemanggilan ulang fungsi composable secara otomatis ketika data atau state yang diamati mengalami perubahan (Android Developers, 2025). Sistem hanya akan memperbarui komponen UI yang terkait langsung dengan data yang

berubah, sehingga efisiensi dan frekuensi pembaruan state menjadi faktor penentu utama terhadap beban kerja pembaruan tampilan.

2.2.2 Pengelolaan State pada Android

State pada aplikasi Android didefinisikan sebagai nilai atau data yang dapat berubah seiring waktu dan memengaruhi perilaku atau tampilan antarmuka pengguna (Movva and Mupparapu, 2026). Pengelolaan state yang terstruktur menerapkan prinsip aliran data satu arah (unidirectional data flow), di mana aliran peristiwa (event) bergerak ke atas menuju pengelola logika, sedangkan aliran data state mengalir ke bawah menuju antarmuka.

Dalam ekosistem Kotlin dan Android modern, state umumnya dikelola menggunakan StateFlow atau MutableStateFlow yang merupakan bagian dari pustaka Kotlin Coroutines (Chauhan et al., 2021). StateFlow menyediakan aliran data yang bersifat observable, aman terhadap thread (thread-safe), dan mempertahankan nilai kondisi terkini untuk dipancarkan kepada komponen antarmuka yang mengamatinya.

2.2.3 Arsitektur Model-View-Intent (MVI)

MVI adalah pola arsitektur perangkat lunak yang dirancang untuk mengatasi kompleksitas pengelolaan kondisi dengan menerapkan aliran data satu arah yang ketat dan deterministik (Vijaywargi and Boddapati, 2024). Pola ini terdiri dari tiga komponen utama:

Intent: Merepresentasikan representasi data dari aksi, peristiwa, atau interaksi yang dipicu oleh pengguna maupun sistem.

Model: Merepresentasikan kondisi aplikasi (UiState) yang bersifat immutable (tidak dapat diubah secara langsung) dan menjadi sumber kebenaran tunggal (single source of truth).

View: Komponen antarmuka pengguna yang bertugas merender representasi state terkini dan meneruskan interaksi pengguna sebagai intent.

Aliran siklus MVI berlangsung secara tertutup: pengguna berinteraksi dengan View, View mengirimkan Intent, logika aplikasi memproses Intent untuk menghasilkan Model (State) baru, dan State baru tersebut dikirim kembali untuk merender ulang View.

2.2.4 Direct State Update MVI

Direct State Update MVI mengacu pada *Pure MVI* yang merupakan salah satu variasi implementasi MVI yang menerapkan mekanisme pembaruan kondisi secara langsung tanpa lapisan perantara (Kostadinov, 2025). Pada pendekatan ini, setiap intent yang diterima oleh komponen pengelola logika (ViewModel) segera ditangani oleh fungsi atau handler spesifik.

Setelah komputasi atau pemrosesan logika selesai dieksekusi, fungsi tersebut langsung memodifikasi dan memperbarui state penampung (misalnya melalui pemanggilan MutableStateFlow.update()). Mekanisme ini menghasilkan alur

eksekusi yang relatif pendek dan langsung, sehingga meminimalkan pembentukan objek perantara dalam memproses interaksi pengguna.

2.2.5 Reducer-Based MVI

Reducer-Based MVI adalah variasi implementasi MVI yang mengadopsi konsep fungsi pereduksi (reducer) dari paradigma pemrograman fungsional, yang dipopulerkan dalam arsitektur pengelolaan state seperti Redux (Redux, 2021; Boduch and Derks, 2023). Dalam landasan teorinya, reducer didefinisikan sebagai fungsi murni (pure function) deterministik yang menerima dua parameter masukan, yaitu state saat ini dan objek representasi aksi, kemudian mengembalikan state baru tanpa memicu efek samping (side effects).

Dalam implementasi Android, intent yang diterima terlebih dahulu diproses untuk menghasilkan representasi perubahan internal, seperti action atau mutation (Vijaywargi and Boddapati, 2024; zen8labs, 2025). Representasi perubahan tersebut kemudian dialirkan ke dalam fungsi reducer terpusat bersama nilai state sebelumnya guna mengkalkulasi dan menginstansiasi objek state baru yang bersifat immutable. Mekanisme ini memusatkan seluruh aturan transisi kondisi aplikasi, namun menambahkan tahapan pemrosesan dan pembuatan objek perantara pada setiap transisi state.

2.2.6 Metrik Responsivitas

Responsivitas adalah rentang waktu total sejak input pengguna dipicu hingga perubahan antarmuka selesai dirender sepenuhnya di layar (Hort et al., 2022). Nilai ini didekomposisi menjadi durasi logika transisi *state* (T_logic) dan durasi *rendering* antarmuka (T_frame):

$$\text{Responsivitas} = T_{logic} + T_{frame} \tag{2.1}$$

Tlogic: waktu CPU memproses *intent* hingga *state* baru terbentuk (ms).

Tframe: waktu subsistem grafis Compose menyelesaikan recomposition, measure, layout, dan draw untuk state tersebut (ms).

Kaitan teori ini dengan rumusan masalah: dekomposisi ini memungkinkan penelitian mengidentifikasi apakah selisih latensi antara kedua varian, jika ada, berasal dari lapisan logika (ViewModel) atau lapisan rendering, sehingga jawaban atas rumusan masalah tidak berhenti pada angka total semata.

2.2.7 Metrik Penggunaan CPU

Penggunaan CPU merupakan ukuran proporsi kapasitas pemrosesan CPU yang digunakan oleh suatu proses selama periode pengamatan. Pada aplikasi Android, penggunaan CPU dapat diamati melalui data *CPU scheduling* yang merekam aktivitas penjadwalan *thread* pada sistem. Penggunaan sumber daya CPU yang tinggi dapat berkaitan dengan penurunan responsivitas antarmuka ketika terjadi persaingan dalam penggunaan sumber daya pemrosesan (Fu et al., 2022).

Pada penelitian ini, penggunaan CPU diukur menggunakan data *scheduling trace* yang direkam oleh Perfetto. Trace dibagi menjadi interval waktu sebesar 50

ms. Pada setiap interval, durasi eksekusi (sched.dur) seluruh *thread* yang termasuk dalam proses aplikasi target dijumlahkan. Nilai tersebut kemudian dinormalisasi terhadap kapasitas CPU yang tersedia pada interval pengamatan, yaitu durasi interval dikalikan dengan jumlah CPU yang terdeteksi dalam *trace*. Persentase penggunaan CPU pada setiap interval dihitung menggunakan persamaan berikut:

$$\text{CPU Usage (\%)} = (\Sigma \text{ sched.dur} / 50\text{ms}) \times 100 \quad (2.2)$$

Σ sched.dur: akumulasi durasi seluruh *slice* penjadwalan *thread* milik proses target selama sesi pengujian (ns).

Kaitan teori ini dengan rumusan masalah: karena tahapan tambahan pada Reducer-Based MVI (Subbab 2.2.5) berupa pemrosesan tambahan pada CPU, metrik ini secara langsung mengukur biaya komputasi dari tahapan tersebut, sehingga menjadi salah satu dari dua ukuran utama yang menjawab rumusan masalah penelitian.

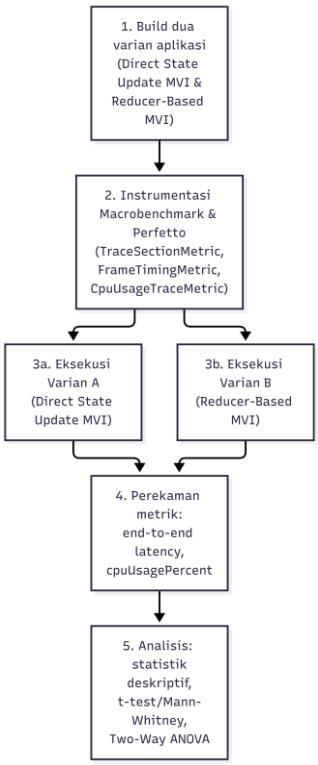
BAB 3 METODOLOGI

3.1 Tipe Penelitian

Penelitian ini merupakan penelitian nonimplementatif dengan tipe analitik, karena bertujuan menganalisis pengaruh mekanisme transisi state (Direct State Update MVI dan Reducer-Based MVI) terhadap performa aplikasi, yaitu responsivitas dan penggunaan CPU.

3.2 Jenis dan Rancangan Penelitian

Strategi yang digunakan adalah eksperimen terkontrol pada perangkat fisik, membandingkan dua arsitektur (Direct State Update MVI dan Reducer-Based MVI). Kedua varian dibangun sebagai dua aplikasi terpisah yang identik pada seluruh komponen non-perlakuan, meliputi tata letak antarmuka, struktur data, logika bisnis, dataset katalog awal, dan spesifikasi perangkat pengujian, sehingga setiap perbedaan performa yang terukur dapat diatribusikan langsung pada perbedaan mekanisme transisi *state*. Perbandingan dilakukan antar kelompok pengukuran independen, karena kedua varian berjalan sebagai *codebase* dan proses aplikasi yang terpisah. Alur pengujian secara keseluruhan, mulai dari penyiapan kedua varian hingga analisis data, ditunjukkan pada Gambar 3.1.

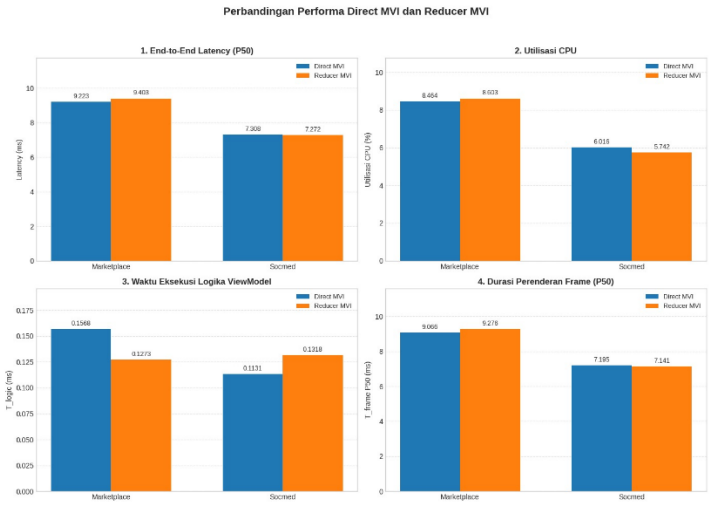


Gambar 3.1 Tahapan Metode Penelitian

3.3 Skenario Eksperimen

Eksperimen dilakukan untuk membandingkan responsivitas dan penggunaan CPU pada aplikasi yang menerapkan Direct State Update MVI dan Reducer-Based MVI. Penelitian menggunakan dua aplikasi dengan karakteristik pengelolaan *state* yang berbeda, yaitu aplikasi marketplace dan sosial media. Perbedaan tingkat *state* tersebut digunakan untuk merepresentasikan kondisi aplikasi dengan beban pengelolaan *state* yang berbeda, sehingga pengaruh mekanisme pengelolaan *state* dapat diamati pada tingkat kompleksitas yang berbeda.

Sebelum eksperimen utama, dilakukan pilot test untuk menentukan tingkat *state* aplikasi berdasarkan jumlah variabel *state* yang dikelola oleh *ViewModel*. Hasil pilot test menunjukkan adanya perbedaan karakteristik performa pada kedua aplikasi sebagaimana ditunjukkan pada Gambar 3.2. Berdasarkan hasil tersebut, aplikasi marketplace dan sosial media dipilih sebagai aplikasi dengan tingkat *state* yang berbeda. Klasifikasi tingkat *state* yang digunakan dalam penelitian disajikan pada Tabel 3.1.



Gambar 3.2 Hasil Pilot Test

Tabel 3.1 Klasifikasi Tingkat State

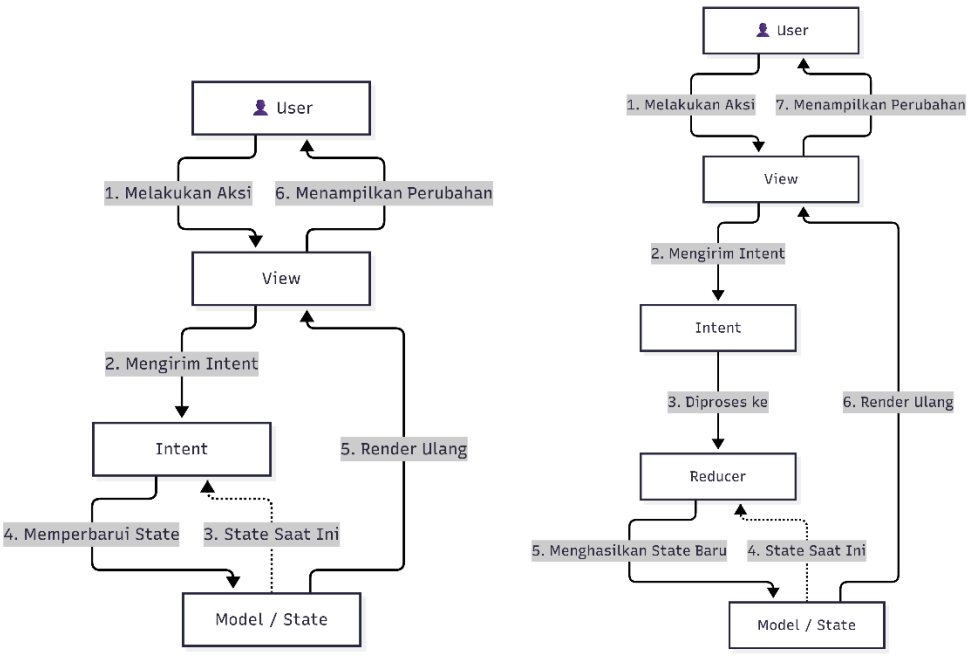
Tingkat State	Jumlah Variabel State	Aplikasi
Low	12	Marketplace
Medium	26	Sosial Media

Eksperimen utama dilakukan pada masing-masing aplikasi dengan dua pendekatan MVI, yaitu Direct State Update MVI dan Reducer-Based MVI. Skenario pengujian dijalankan secara otomatis menggunakan Android Macrobenchmark

dan UI Automator dengan rangkaian interaksi yang telah ditentukan. Setiap pendekatan diuji sebanyak 30 kali untuk memperoleh data pengukuran. Perbedaan mekanisme pengelolaan state pada kedua tipe ditunjukkan pada Gambar 3.3.

Tabel 3.2 Skenario Eksperimen

Aplikasi	Tingkat State	Skenario Pengujian	Pendekatan MVI	Jumlah Pengujian
Marketplace	Low	Pemilihan produk, penambahan dan pengurangan kuantitas, pengguliran daftar produk, penerapan voucher.	Direct State Update MVI, Reducer-Based MVI	30
Sosial Media	Medium	Pengguliran unggahan, menyukai unggahan, membuka komentar, menyukai komentar, *filter* unggahan.	Direct State Update MVI, Reducer-Based MVI	30



Gambar 3.3 Dua Tipe MVI

Berdasarkan skenario eksperimen pada Tabel 3.2, penelitian ini merumuskan hipotesis untuk menguji apakah perbedaan mekanisme pengelolaan *state* antara Direct State Update MVI dan Reducer-Based MVI menghasilkan perbedaan performa. Hipotesis penelitian mencakup dua variabel terikat yang diukur, yaitu responsivitas dan penggunaan CPU. Rincian hipotesis yang akan diuji pada hasil eksperimen disajikan pada Tabel 3.3.

Tabel 3.3 Hipotesis Penelitian

Kode	Skenario Pengujian	Variabel
H1	Terdapat perbedaan responsivitas antara aplikasi yang menerapkan Direct State Update MVI dan Reducer-Based MVI.	Responsivitas
H2	Terdapat perbedaan penggunaan CPU antara aplikasi yang menerapkan Direct State Update MVI dan Reducer-Based MVI	Penggunaan CPU

3.4 Metode dan Metrik Pengukuran

Pengukuran performa dilakukan menggunakan kerangka kerja AndroidX Macrobenchmark dengan memanfaatkan penanda jejak sistem (system tracing) Perfetto. Metrik yang dikumpulkan mencakup dua parameter utama penelitian, yaitu latensi responsivitas dan penggunaan CPU.

3.4.1 Pengukuran Responsivitas

Responsivitas dalam penelitian ini merupakan keseluruhan waktu yang diperlukan aplikasi sejak interaksi pengguna dipicu hingga perubahan antarmuka sebagai respons terhadap interaksi tersebut selesai diproses dan direnderer. Pengukuran dilakukan dengan memandang proses tersebut sebagai gabungan antara waktu pemrosesan logika transisi state dan waktu perenderan antarmuka.

Waktu pemrosesan logika transisi state diukur menggunakan TraceSectionMetric dengan penanda khusus yang membedakan proses Direct State Update MVI dan Reducer-Based MVI. Pengukuran ini merepresentasikan waktu yang diperlukan aplikasi untuk menerima intent, menjalankan logika transaksi, serta menghasilkan state baru. Waktu perenderan antarmuka diukur menggunakan FrameTimingMetric yang merekam waktu pemrosesan frame setelah perubahan state diterapkan pada antarmuka Jetpack Compose.

Nilai responsivitas diperoleh dengan menggabungkan waktu pemrosesan logika transisi state dan waktu perenderan antarmuka. Nilai tersebut kemudian dianalisis untuk membandingkan karakteristik respons aplikasi antara Direct State Update MVI dan Reducer-Based MVI. Selain nilai pusat, persentil ke-50 dan persentil ke-95 digunakan untuk menggambarkan kondisi tipikal dan kondisi dengan beban rendering yang lebih tinggi.

3.5.2 Pengukuran Penggunaan CPU

Penggunaan CPU diukur menggunakan CpuUsageTraceMetric berdasarkan data *CPU scheduling trace* yang direkam oleh Perfetto. Data *scheduling* difilter berdasarkan *package name* aplikasi target, kemudian dikelompokkan ke dalam interval waktu 50 ms. Pada setiap interval, durasi sched.dur dari seluruh *thread* aplikasi dijumlahkan dan dinormalisasi terhadap jumlah CPU yang terdeteksi untuk memperoleh persentase penggunaan CPU.

Nilai penggunaan CPU dari seluruh interval kemudian diurutkan dan dihitung nilai mediannya. Nilai median tersebut digunakan sebagai hasil pengukuran cpuUsagePercent untuk setiap pengujian. Hasil pengukuran pada Direct State Update MVI dan Reducer-Based MVI selanjutnya dibandingkan pada kondisi pengujian yang sama.

3.5 Lingkungan dan Instrumen Pengujian

Pengujian dilakukan pada lingkungan eksperimen yang terkontrol menggunakan satu unit perangkat smartphone fisik Samsung Galaxy A24 dengan RAM 8 GB dan sistem operasi Android 15 API level 35. Aplikasi yang diuji dikembangkan menggunakan bahasa pemrograman Kotlin dan Jetpack Compose dengan pustaka Kotlin Coroutines dan Kotlin Flow yang digunakan dalam mekanisme pengelolaan state. Kedua pendekatan MVI dijalankan pada perangkat dan lingkungan perangkat lunak yang sama untuk meminimalkan pengaruh faktor eksternal terhadap hasil pengukuran.

Instrumen pengujian terdiri atas AndroidX Macrobenchmark, UI Automator, dan Perfetto. AndroidX Macrobenchmark digunakan untuk menjalankan dan mengulangi pengujian performa, UI Automator digunakan untuk mengotomatisasi interaksi pengguna, sedangkan Perfetto digunakan untuk memperoleh data system tracing dan menganalisis aktivitas pemrosesan aplikasi. Pada benchmark, aplikasi dijalankan menggunakan mode kompilasi penuh dan StartupMode.WARM untuk menjaga kondisi eksekusi tetap konsisten. Implementasi benchmark menggunakan FrameTimingMetric, TraceSectionMetric, dan CpuUsageTraceMetric untuk memperoleh data performa yang diperlukan.

3.6 Teknik Analisis Data

Data hasil pengukuran performa diolah dan dianalisis menggunakan dua tingkatan analisis statistik, yaitu analisis statistik deskriptif dan analisis statistik inferensial pada taraf signifikansi $\alpha = 0{,}05$. Analisis dilakukan terhadap dua metrik penelitian, yaitu responsivitas sebagai metrik utama dan *CPU usage* sebagai metrik pendukung.

Analisis statistik deskriptif dilakukan dengan menyajikan data dalam tabel komparatif yang memuat ukuran pemusatan dan persebaran data. Untuk metrik responsivitas dan penggunaan CPU, analisis mencakup nilai rata-rata (*mean*), nilai tengah (*median*), standar deviasi, nilai minimum dan maksimum, serta persentil ke-50 (P50) dan persentil ke-95 (P95).

Analisis statistik inferensial dilakukan untuk mengetahui apakah terdapat perbedaan yang signifikan pada *responsivitas* dan *CPU usage* antara *Direct State Update MVI* dan *Reducer-Based MVI*. Karena kedua pendekatan diuji pada tingkat beban kerja yang sama, pengujian dilakukan dengan mempertimbangkan pasangan data dari kondisi pengujian yang sesuai. Uji normalitas Shapiro-Wilk digunakan untuk menentukan distribusi data. Apabila data memenuhi asumsi normalitas, perbedaan antara kedua pendekatan dianalisis menggunakan *Paired Sample t-Test*, sedangkan apabila asumsi normalitas tidak terpenuhi digunakan uji non-parametrik *Wilcoxon Signed-Rank Test*. Besarnya perbedaan juga dianalisis menggunakan *effect size* untuk mengetahui kekuatan perbedaan performa antara kedua pendekatan.

Hasil analisis statistik deskriptif dan inferensial selanjutnya digunakan sebagai dasar dalam menginterpretasikan perbedaan performa antara *Direct State Update MVI* dan *Reducer-Based MVI* serta menjawab rumusan masalah penelitian.

DAFTAR REFERENSI

CommandLinux, 2026. Android global market share statistics 2026. [online] Available at: <https://commandlinux.com/android/android-global-market-share-statistics/> [Accessed 14 August 2026].

Kwiatkowski, A. and Smołka, J., 2025. Performance analysis of Jetpack Compose components in mobile applications. Journal of Computer Sciences Institute, 37, pp.391-398. https://doi.org/10.35784/jcsi.7914.

Android Developers, 2025. What developers are saying about Jetpack Compose. [online] Available at: <https://developer.android.com/develop/ui/compose/adopt#what-developers-are-saying> [Accessed 19 May 2025].

Chauhan, K., Kumar, S., Sethia, D. and Alam, M.N., 2021. Performance analysis of Kotlin coroutines on Android in a model-view-intent architecture pattern. In: 2021 2nd International Conference for Emerging Technology (INCET). Belgaum, India, 21-23 May 2021. Piscataway: IEEE, pp.1-6. https://doi.org/10.1109/incet51464.2021.9456197.

Kostadinov, D., 2025. Pure MVI, MVI with reducer, and MVI with state machine. ProAndroidDev. [online] Available at: <https://proandroiddev.com/pure-mvi-mvi-with-reducer-and-mvi-with-state-machine-43a0ec4b629f> [Accessed 14 August 2026].

Redux, 2021. Structuring reducers. [online] Available at: <https://redux.js.org/usage/structuring-reducers/structuring-reducers> [Accessed 14 August 2026].

Boduch, A. and Derks, R., 2023. React and React Native: Build modern cross-platform web and mobile applications using React and React Native. 4th ed. Birmingham: Packt Publishing.

Vijaywargi, A. and Boddapati, U.K., 2024. Architectural patterns in Android development: comparing MVP, MVVM, and MVI. International Journal for Research in Applied Science and Engineering Technology, 12(4), pp.4611-4616. https://doi.org/10.22214/ijraset.2024.60762.

GitHub Community, 2026. Jetpack Compose recomposition issues with StateFlow in large screens (Android Studio + Kotlin). [online] Available at: https://github.com/orgs/community/discussions/186785 [Accessed 3 October 2026]

Reddit, 2025. Shopee Cart section slow. [online] Available at: https://www.reddit.com/r/ShopeePH/comments/1n92eyz/shopee_cart_section_slow/ [Accessed 3 October 2026].

Samsung Community, 2025. Instagram lagging. [online] Available at: https://eu.community.samsung.com/t5/galaxy-s24-series/instagram-lagging/m-p/11396223/highlight/true [Accessed 3 October 2026].

Paramadani, R.B., Akbar, M.A. and Pinandito, A., 2023. User interface rendering time in Android applications: revealing the effects of design patterns. In: International Conference on Sustainable Information Engineering and Technology (SIET 2023). Badung, Bali, Indonesia, 24–25 October 2023. New York: ACM, pp.1-6. https://doi.org/10.1145/3626641.3626948.

zen8labs, 2025. How to master MVI with reducers in Android. Medium. [online] Available at: <https://medium.com/zen8labs/how-to-master-mvi-with-reducers-in-android-38a97f3403b4> [Accessed 14 August 2026].

Lin, H., Liu, C., Li, Z., Qian, F., Li, M., Xiong, P. and Liu, Y., 2024. Aging or glitching? What leads to poor Android responsiveness and what can we do about it? IEEE Transactions on Mobile Computing, 23(2), pp.1521-1533. https://doi.org/10.1109/TMC.2023.3237716.

Chanddru, V., 2025. Optimizing mobile app performance to enhance user satisfaction and engagement. World Journal of Advanced Engineering Technology and Sciences, 15(1), pp.828-838. https://doi.org/10.30574/wjaets.2025.15.1.0303.

Android Developers, 2025. Overview of memory management. [online] Available at: <https://developer.android.com/topic/performance/memory-overview> [Diakses 16 September 2026].

Hort, M., Kechagia, M., Sarro, F. and Harman, M., 2022. A survey of performance optimization for mobile applications. IEEE Transactions on Software Engineering, 48(8), pp.2879-2904. https://doi.org/10.1109/TSE.2021.3071193.

Android Developers, 2025. Capture Macrobenchmark metrics. [online] Available at: <https://developer.android.com/topic/performance/benchmarking/macrobenchmark-metrics> [Diakses 16 September 2026].

Fu, J., Wang, Y., Zhou, Y. and Wang, X., 2022. How resource utilization influences UI responsiveness of Android software. Information and Software Technology, 141, p.106728. https://doi.org/10.1016/j.infsof.2021.106728.

Movva, C.K. and Mupparapu, H.K., 2026. MVVM architectural patterns for enterprise Android kiosk applications: state management and lifecycle handling. International Journal of Emerging Trends in Computer Science and Information Technology, 7(2), pp.320-330. https://doi.org/10.63282/3050-9246.IJETCSIT-V7I2P139.